TABLE OF CONTENT

SL.NO	Question	Page No
1	Analyze a dataset of your choice (e.g., Iris dataset, Titanic dataset) to identify patterns and relationships between variables.	3
2	Prepare a dataset for a machine learning task. Implement techniques like imputation, one-hot encoding, and standardization/normalization.	7
3	Build a k-NN classifier to predict the species of iris flowers based on sepal and petal measurements. Experiment with different values of 'k'.	10
4	Build a linear regression model to predict housing prices based on features like area, number of bedrooms, and location.	12
5	Build a logistic regression model to predict whether a customer will click on an ad based on demographics and browsing history.	14
6	Evaluate and compare linear regression, polynomial regression, and Ridge/Lasso regression for a prediction task of your choice.	16
7	Build a decision tree classifier to predict customer churn based on service usage patterns and account information. Visualize the tree and analyze feature importance.	18
8	Build a Random Forest classifier for a classification task. Tune hyperparameters (e.g., number of trees) to optimize performance.	21
9	Build a spam email classifier using the Naive Bayes algorithm.	24
10	Segment customers based on their purchasing behavior (e.g., RFM: Recency, Frequency, Monetary value) using k-means clustering. Visualize the clusters.	26

Program 1: Question

Each Program should include,

- Complete Question
- Program Code.
- Output with explanation of the Output
- Possible Viva Questions

Machine Learning Laboratory SEMESTER – V			
Course Code	**M23BCIL504**	CIE Marks	**50**
Number of Lecture Hours/Week(L: T: P: S)	**0:0:2:0**	SEE Marks	**50**
Total Number of Lecture Hours	**24 Hours Practical**	Total Marks	**100**
Credits	**01**	Exam Hours	**03**

1: Data Exploration and Visualization
Aim: Familiarize with basic data loading, cleaning, and visualization techniques.
Question: Analyze a dataset of your choice (e.g., Iris dataset, Titanic dataset) to identify patterns and relationships between variables.
Dataset:
Iris: https://www.kaggle.com/datasets/uciml/iris
Titanic: https://www.kaggle.com/c/titanic.

2: Data pre-processing Pipeline
Aim: Build a data pre-processing pipeline to handle missing values, categorical data, and feature scaling.
Question: Prepare a dataset for a machine learning task. Implement techniques like imputation, one-hot encoding, and standardization/normalization.
Dataset: Use the same dataset from Program 1 or a different one with similar characteristics.

3: Implementing k-Nearest Neighbors
Aim: Understand and implement the k-NN algorithm for classification.
Question: Build a k-NN classifier to predict the species of iris flowers based on sepal and petal measurements. Experiment with different values of 'k'**.**
Dataset: Iris dataset.

4: Linear Regression for Prediction
Aim: Apply linear regression to predict a continuous target variable.
Question: Build a linear regression model to predict housing prices based on features like area, number of bedrooms, and location.
Dataset:
Boston Housing: https://www.kaggle.com/datasets/vikrishnan/boston-house-prices
California Housing**:** https://scikit-learn.org/stable/datasets/real_world.html#california-housing-dataset.

5: Logistic Regression for Classification
Aim: Use logistic regression for binary or multi-class classification.
Question: Build a logistic regression model to predict whether a customer will click on an ad based on demographics and browsing history.
Dataset: You can simulate this type of dataset or find related advertising datasets on Kaggle.

6: Comparing Regression Models
Aim: Compare the performance of different regression models on a given dataset.
Question: Evaluate and compare linear regression, polynomial regression, and Ridge/Lasso regression for a prediction task of your choice.
Dataset: Use a dataset suitable for regression, potentially from previous programs.

7: Decision Tree Classification
Aim: Visualize and interpret decision tree models.
Question: Build a decision tree classifier to predict customer churn based on service usage patterns and account information. Visualize the tree and analyze feature importance.
Dataset: You can simulate a churn dataset or search for "telecom churn" datasets.

8: Ensemble Methods - Random Forest
Aim: Apply the Random Forest algorithm and evaluate its performance.
Question: Build a Random Forest classifier for a classification task. Tune hyperparameters (e.g., number of trees) to optimize performance.
Dataset: Choose a dataset suitable for classification, possibly from a previous program.

9: Naive Bayes for Text Classification - Aim: Implement the Naive Bayes algorithm for text data.
Question: Build a spam email classifier using the Naive Bayes algorithm.
Dataset: Spambase: https://archive.ics.uci.edu/ml/datasets/Spambase
Other text datasets: https://www.kaggle.com/datasets?tags=text

10: Customer Segmentation with K-Means Clustering **Aim:** Apply clustering to group similar data points.
Question: Segment customers based on their purchasing behavior (e.g., RFM: Recency, Frequency, Monetary value) using k-means clustering. Visualize the clusters.
Dataset: Simulate a customer purchase dataset or use a retail dataset with transaction history.

1: Data Exploration and Visualization

Aim: Familiarize with basic data loading, cleaning, and visualization techniques.
Question: Analyze a dataset of your choice (e.g., Iris dataset, Titanic dataset) to identify patterns and relationships between variables.
Dataset:
Iris: https://www.kaggle.com/datasets/uciml/iris
Titanic: https://www.kaggle.com/c/titanic.

CODE:

```
# 1. Import necessary libraries
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris

# 2. Load the Iris dataset
iris = load_iris()
df = pd.DataFrame(data=iris.data, columns=iris.feature_names)
df['species'] = iris.target
df['species'] = df['species'].map({i: species for i, species in enumerate(iris.target_names)})

# 3. Preview the data
print("First 5 rows of dataset:")
display(df.head())

# 4. Dataset summary
print("\nDataset Info:")
df.info()

print("\nSummary Statistics:")
display(df.describe())

# 5. Visualizations
sns.pairplot(df, hue="species")
plt.suptitle("Pairplot of Iris Features by Species", y=1.02)
plt.show()

# Correlation heatmap
plt.figure(figsize=(8, 6))
sns.heatmap(df.iloc[:, :-1].corr(), annot=True, cmap='coolwarm')
plt.title("Correlation Matrix")
plt.show()

# Boxplots for each feature
for feature in iris.feature_names:
    plt.figure(figsize=(6, 4))
```

```
sns.boxplot(x='species', y=feature, data=df)
plt.title(f"{feature.capitalize()} by Species")
plt.show()
```

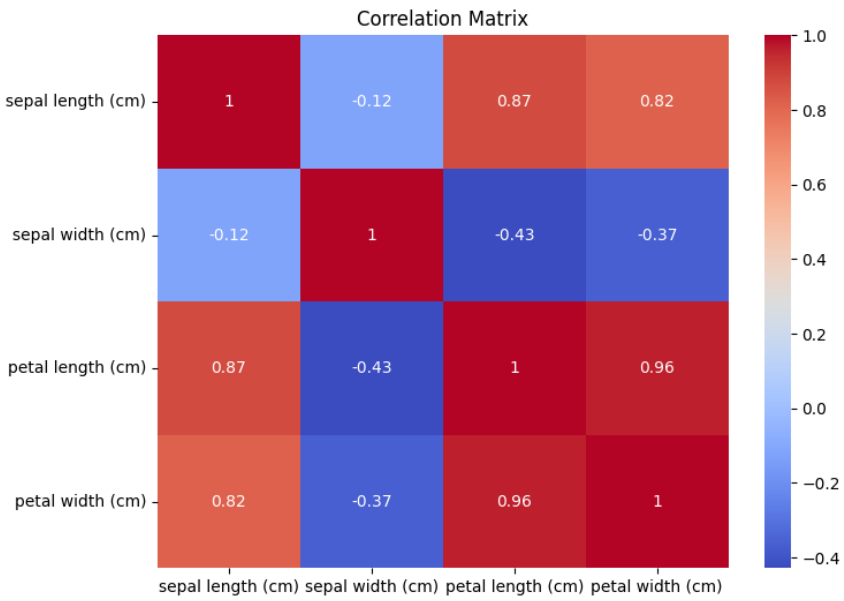
OUTPUT:

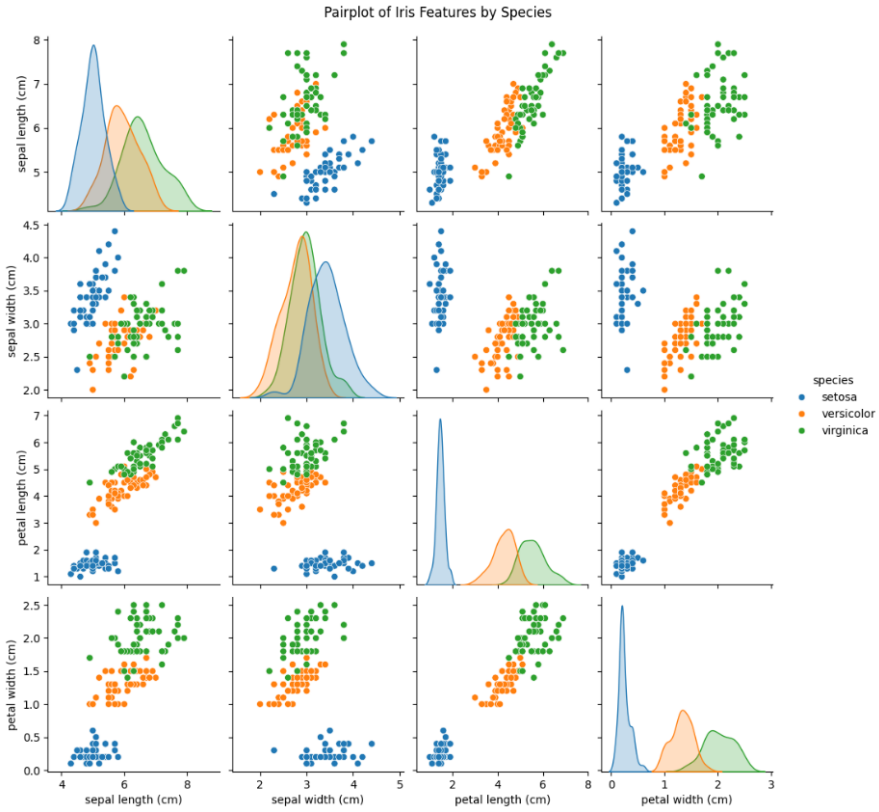
```
First 5 rows of dataset:
```

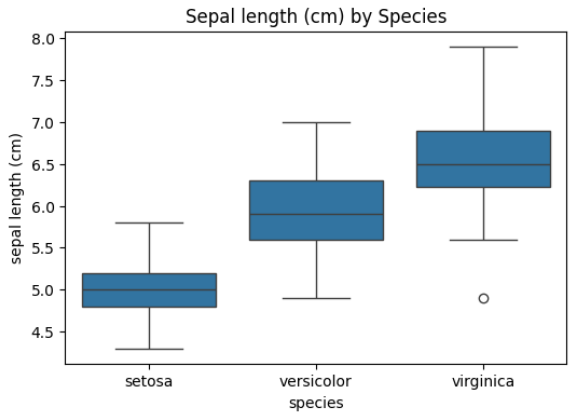
	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	species
0	5.1	3.5	1.4	0.2	setosa
1	4.9	3.0	1.4	0.2	setosa
2	4.7	3.2	1.3	0.2	setosa
3	4.6	3.1	1.5	0.2	setosa
4	5.0	3.6	1.4	0.2	setosa

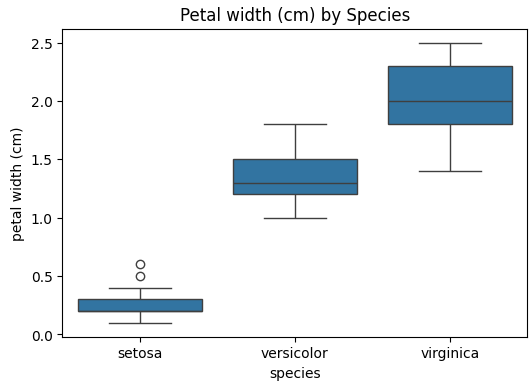
```
Dataset Info:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 150 entries, 0 to 149
Data columns (total 5 columns):
 #   Column             Non-Null Count  Dtype
---  ------             --------------  -----
 0   sepal length (cm)  150 non-null    float64
 1   sepal width (cm)   150 non-null    float64
 2   petal length (cm)  150 non-null    float64
 3   petal width (cm)   150 non-null    float64
 4   species            150 non-null    object
dtypes: float64(4), object(1)
memory usage: 6.0+ KB

Summary Statistics:
```

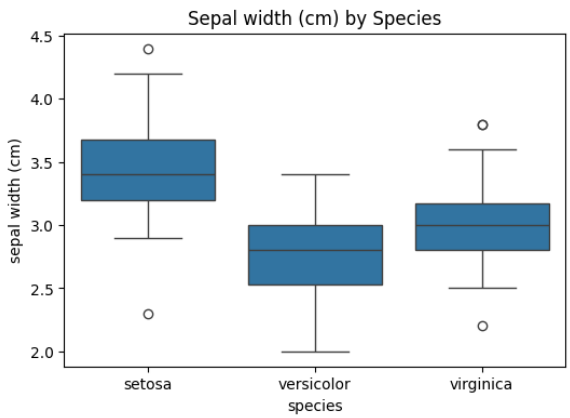
	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)
count	150.000000	150.000000	150.000000	150.000000
mean	5.843333	3.057333	3.758000	1.199333
std	0.828066	0.435866	1.765298	0.762238
min	4.300000	2.000000	1.000000	0.100000
25%	5.100000	2.800000	1.600000	0.300000
50%	5.800000	3.000000	4.350000	1.300000
75%	6.400000	3.300000	5.100000	1.800000
max	7.900000	4.400000	6.900000	2.500000

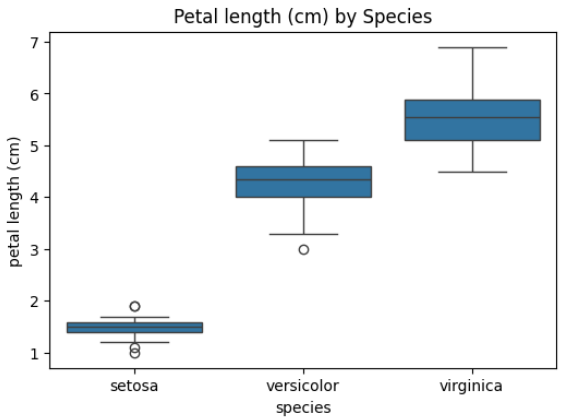












Possible Viva Questions:

Topic: Loading, cleaning, and visualizing datasets like Iris or Titanic.

1. What is the primary difference between a Pandas Series and a DataFrame?
2. Which Python libraries are standard for data visualization, and how do they differ (e.g., Matplotlib vs. Seaborn)?
3. How do you read a CSV file in Pandas, and what parameters might you need to change if the delimiter is not a comma?
4. What is the purpose of the .describe() method in Pandas?
5. Explain the difference between a histogram and a bar chart.
6. How would you identify outliers using a box plot?
7. What does a correlation matrix tell you about the relationships between variables?
8. Why is data cleaning considered the most time-consuming part of a data science project?
9. What is the significance of the null or NaN value in a dataset?
10. How do scatter plots help in identifying relationships between two continuous variables?
11. What is the difference between univariate and bivariate analysis?
12. Why did you choose the specific visualization technique you used for this experiment?

2: Data pre-processing Pipeline

Aim: Build a data pre-processing pipeline to handle missing values, categorical data, and feature scaling.
Question: Prepare a dataset for a machine learning task. Implement techniques like imputation, one-hot encoding, and standardization/normalization.
Dataset: Use the same dataset from Program 1 or a different one with similar characteristics.

CODE:

```
import pandas as pd
import numpy as np
from sklearn.datasets import load_iris
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import OneHotEncoder, StandardScaler
# Load Iris dataset
iris = load_iris()
X = pd.DataFrame(iris.data, columns=iris.feature_names)

# Simulate missing values (for demonstration)
np.random.seed(0)
X.loc[X.sample(frac=0.1).index, 'sepal length (cm)'] = np.nan

# Simulate a categorical column
X['flower_type'] = np.random.choice(['Type A', 'Type B', 'Type C'], size=len(X))

# ----------------------------
# 1. Impute Missing Values
# ----------------------------
imputer = SimpleImputer(strategy='mean')
X_numeric_imputed = imputer.fit_transform(X[iris.feature_names])
X_numeric_imputed_df = pd.DataFrame(X_numeric_imputed, columns=iris.feature_names)

print("After Imputation (Missing Values Handled):")
print(X_numeric_imputed_df.head())
# ----------------------------
# 2. One-Hot Encode Categorical Column
# ----------------------------
encoder = OneHotEncoder(handle_unknown='ignore')
X_categorical_encoded = encoder.fit_transform(X[['flower_type']])
encoded_df = pd.DataFrame(X_categorical_encoded.toarray(),
        columns=encoder.get_feature_names_out())

print("\n After One-Hot Encoding (Categorical Data Handled):")
print(encoded_df.head())
# ----------------------------
# 3. Standardize Numeric Features
```

```
# ----------------------------
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X_numeric_imputed_df)
X_scaled_df = pd.DataFrame(X_scaled, columns=iris.feature_names)

print("\n After Standardization (Feature Scaling Applied):")
print(X_scaled_df.head())
# ----------------------------
# Final Processed Dataset
# ----------------------------
X_processed = pd.concat([X_scaled_df, encoded_df], axis=1)

print("\n Final Preprocessed Dataset:")
print(X_processed.head())
print("\n Final shape:", X_processed.shape)
```

OUTPUT:

```
✅ After Imputation (Missing Values Handled):
   sepal length (cm)  sepal width (cm)  petal length (cm)  petal width (cm)
0                5.1               3.5                1.4               0.2
1                4.9               3.0                1.4               0.2
2                4.7               3.2                1.3               0.2
3                4.6               3.1                1.5               0.2
4                5.0               3.6                1.4               0.2


✅ After One-Hot Encoding (Categorical Data Handled):
   flower_type_Type A  flower_type_Type B  flower_type_Type C
0                 1.0                 0.0                 0.0
1                 1.0                 0.0                 0.0
2                 0.0                 0.0                 1.0
3                 0.0                 0.0                 1.0
4                 1.0                 0.0                 0.0

✅ After Standardization (Feature Scaling Applied):
   sepal length (cm)  sepal width (cm)  petal length (cm)  petal width (cm)
0          -0.894017          1.019004          -1.340227         -1.315444
1          -1.144675         -0.131979          -1.340227         -1.315444
2          -1.395334          0.328414          -1.397064         -1.315444
3          -1.520664          0.098217          -1.283389         -1.315444
4          -1.019346          1.249201          -1.340227         -1.315444

✅ Final Preprocessed Dataset:
   sepal length (cm)  sepal width (cm)  petal length (cm)  petal width (cm)  \
0          -0.894017          1.019004          -1.340227         -1.315444
1          -1.144675         -0.131979          -1.340227         -1.315444
2          -1.395334          0.328414          -1.397064         -1.315444
3          -1.520664          0.098217          -1.283389         -1.315444
4          -1.019346          1.249201          -1.340227         -1.315444

   flower_type_Type A  flower_type_Type B  flower_type_Type C
0                 1.0                 0.0                 0.0
1                 1.0                 0.0                 0.0
2                 0.0                 0.0                 1.0
3                 0.0                 0.0                 1.0
4                 1.0                 0.0                 0.0

🔚 Final shape: (150, 7)
```

Possible Viva Questions:

Topic: Handling missing values, encoding, and scaling.

1. What is a "Pipeline" in Scikit-Learn and why is it useful?
2. What are the different strategies for handling missing data (imputation)?
3. Explain the difference between Label Encoding and One-Hot Encoding. When should you use each?
4. What is the "Curse of Dimensionality," and how does One-Hot Encoding contribute to it?
5. What is the difference between Standardization (Z-score) and Normalization (Min-Max scaling)?
6. Why is feature scaling important for algorithms like k-NN or SVM?
7. What does fit_transform() do, and how is it different from transform()?
8. Why must you fit your scaler on the training set only and not the test set?
9. How do you handle categorical variables that have an inherent order (ordinal data)?
10. What is "Data Leakage" in the context of preprocessing?
11. Can you explain what the SimpleImputer class does?
12. How does the presence of outliers affect Min-Max scaling vs. Standardization?

3: Implementing k-Nearest Neighbors

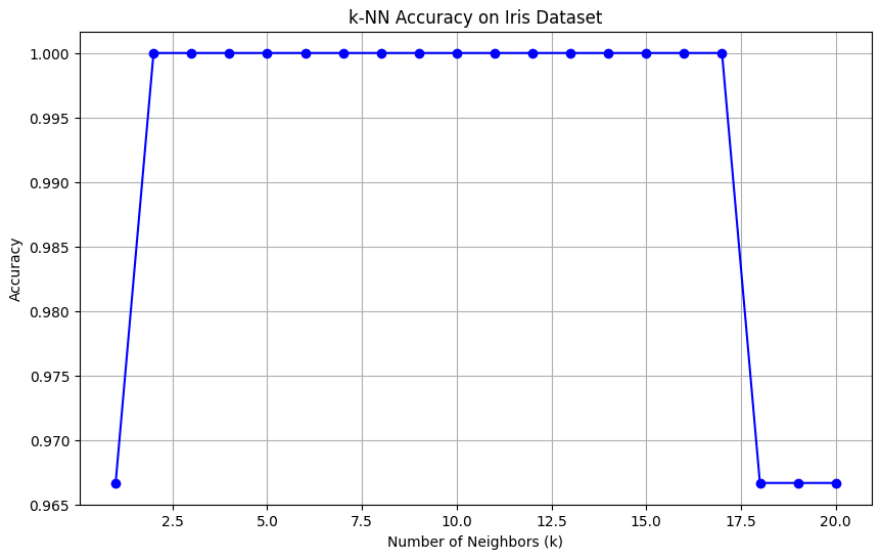
Aim: Understand and implement the k-NN algorithm for classification.
Question: Build a k-NN classifier to predict the species of iris flowers based on sepal and petal measurements. Experiment with different values of 'k'.
Dataset: Iris dataset.

CODE:

```
# Import required libraries
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn import datasets
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score

# Load the Iris dataset
iris = datasets.load_iris()
X = iris.data
y = iris.target
target_names = iris.target_names

# Standardize features
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# Split into training and testing sets (80% train, 20% test)
X_train, X_test, y_train, y_test = train_test_split(X_scaled, y, test_size=0.2, random_state=42)

# Try different values of k and record accuracy
k_values = list(range(1, 21))
accuracies = []

for k in k_values:
    knn = KNeighborsClassifier(n_neighbors=k)
    knn.fit(X_train, y_train)
    y_pred = knn.predict(X_test)
    acc = accuracy_score(y_test, y_pred)
    accuracies.append(acc)
    print(f"k={k}: Accuracy = {acc:.2f}")

# Plot accuracy vs k
plt.figure(figsize=(10, 6))
plt.plot(k_values, accuracies, marker='o', linestyle='-', color='b')
plt.title("k-NN Accuracy on Iris Dataset")
plt.xlabel("Number of Neighbors (k)")
plt.ylabel("Accuracy")
plt.grid(True)
plt.show()
```

OUTPUT:

```
k=1: Accuracy = 0.97
k=2: Accuracy = 1.00
k=3: Accuracy = 1.00
k=4: Accuracy = 1.00
k=5: Accuracy = 1.00
k=6: Accuracy = 1.00
k=7: Accuracy = 1.00
k=8: Accuracy = 1.00
k=9: Accuracy = 1.00
k=10: Accuracy = 1.00
k=11: Accuracy = 1.00
k=12: Accuracy = 1.00
k=13: Accuracy = 1.00
k=14: Accuracy = 1.00
k=15: Accuracy = 1.00
k=16: Accuracy = 1.00
k=17: Accuracy = 1.00
k=18: Accuracy = 0.97
k=19: Accuracy = 0.97
k=20: Accuracy = 0.97
```



Possible Viva Questions:
Topic: Classification using k-NN on the Iris dataset.

1. Why is k-NN called a "Lazy Learner"?
2. How does the value of 'k' affect the bias-variance tradeoff?
3. What happens if you choose $k=1$? Is the model more likely to overfit or underfit?
4. Why is it generally recommended to choose an odd number for 'k' in binary classification?
5. Which distance metric did you use (Euclidean, Manhattan, etc.), and why?
6. How do you calculate Euclidean distance between two points in n-dimensional space?
7. Does k-NN perform well on large datasets? Why or why not?
8. How does feature scaling affect the performance of k-NN?
9. What is the difference between k-NN classification and k-NN regression?
10. What is the computational complexity of the testing phase in k-NN?
11. How does the "Curse of Dimensionality" impact k-NN?
12. Can k-NN derive a decision boundary that is non-linear?

4: Linear Regression for Prediction
Aim: Apply linear regression to predict a continuous target variable. Question: Build a linear regression model to predict housing prices based on features like area, number of bedrooms, and location.
Dataset:
Boston Housing: https://www.kaggle.com/datasets/vikrishnan/boston-house-prices
California Housing: https://scikit-learn.org/stable/datasets/real_world.html#california-housing-dataset.

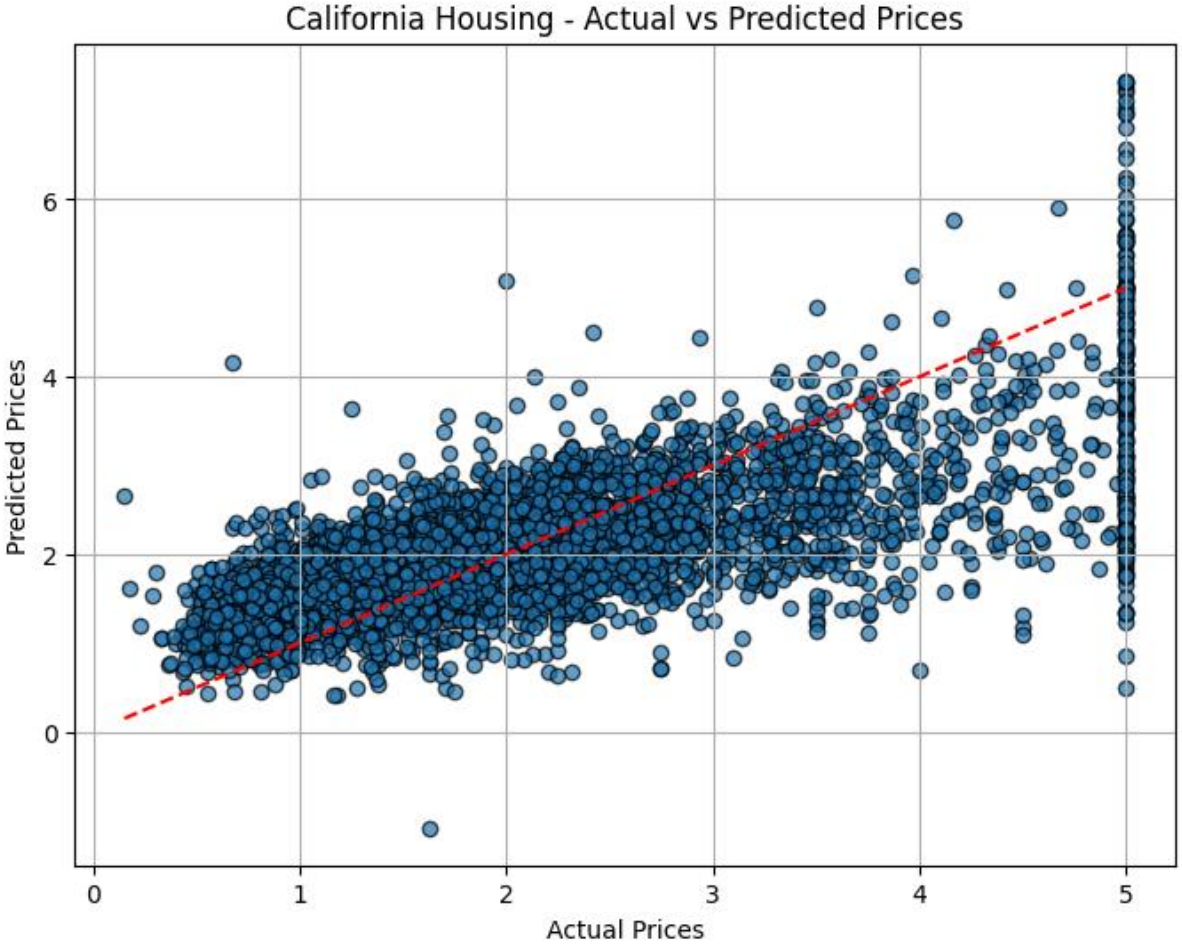
CODE:

```
# Import libraries
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
# Import fetch_california_housing instead of load_boston
from sklearn.datasets import fetch_california_housing
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error, r2_score
from sklearn.preprocessing import StandardScaler

# Load dataset - Using California Housing as suggested by the error message
housing = fetch_california_housing()
X = pd.DataFrame(housing.data, columns=housing.feature_names)
y = pd.Series(housing.target, name='MedHouseVal') # Target name is 'MedHouseVal' in California Housing

# Select relevant features
# Adjust feature selection for California Housing dataset.
# We'll use MedInc (Median Income ~ a proxy for wealth/location), HouseAge, and AveRooms
 #(Average number of rooms ~ related to size)
X_selected = X[['MedInc', 'HouseAge', 'AveRooms']]

# Standardize features
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X_selected)

# Train-test split
X_train, X_test, y_train, y_test = train_test_split(X_scaled, y, test_size=0.2, random_state=42)

# Train model
model = LinearRegression()
model.fit(X_train, y_train)

# Predict
y_pred = model.predict(X_test)

# Evaluate
print("Mean Squared Error:", mean_squared_error(y_test, y_pred))
print("R² Score:", r2_score(y_test, y_pred))

# Plot
```

```
plt.figure(figsize=(8, 6))
plt.scatter(y_test, y_pred, edgecolors='k', alpha=0.7)
# Adjust plot range for California Housing data if necessary, or keep dynamic
plt.plot([min(y_test), max(y_test)], [min(y_test), max(y_test)], 'r--')
plt.xlabel("Actual Prices")
plt.ylabel("Predicted Prices")
plt.title("California Housing - Actual vs Predicted Prices") # Update title
plt.grid(True)
plt.show()
```

OUTPUT:

Mean Squared Error: 0.6589108649336336
R^2 Score: 0.49717158850807075



Possible Viva Questions:
Topic: Predicting housing prices.

1. What is the hypothesis function for Linear Regression?
2. What is the geometric interpretation of the coefficient (slope) and intercept?
3. What is the Cost Function (Mean Squared Error) in Linear Regression?
4. Explain the Gradient Descent algorithm in simple terms.
5. What are the key assumptions of Linear Regression?
6. What is the difference between Simple Linear Regression and Multiple Linear Regression?
7. What metric is used to evaluate the accuracy of a regression model?
8. What is R^2 (R-squared) and what does a value of 0.8 imply?
9. How do outliers affect the "Line of Best Fit"?
10. What is Multicollinearity, and why is it a problem in regression?

5: Logistic Regression for Classification

Aim: Use logistic regression for binary or multi-class classification.
Question: Build a logistic regression model to predict whether a customer will click on an ad based on demographics and browsing history.
Dataset: You can simulate this type of dataset or find related advertising datasets on Kaggle.

CODE:

```
import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import classification_report, confusion_matrix, accuracy_score
import matplotlib.pyplot as plt
import seaborn as sns

# 1. Simulate Data
np.random.seed(42)
n_samples = 1000

# Simulated features
age = np.random.randint(18, 70, n_samples)
income = np.random.randint(20000, 150000, n_samples)
browsing_time = np.random.normal(5, 2, n_samples)  # hours/day
clicked = (0.3 * (age < 30) + 0.4 * (income < 50000) + 0.5 * (browsing_time > 5) +
          np.random.normal(0, 0.2, n_samples)) > 0.8
clicked = clicked.astype(int)

# Create DataFrame
df = pd.DataFrame({
    'Age': age,
    'Annual_Income': income,
    'Browsing_Hours': browsing_time,
    'Clicked_on_Ad': clicked
})

# 2. Split Data
X = df[['Age', 'Annual_Income', 'Browsing_Hours']]
y = df['Clicked_on_Ad']
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# 3. Logistic Regression Model
model = LogisticRegression()
model.fit(X_train, y_train)

# 4. Prediction and Evaluation
y_pred = model.predict(X_test)

print("Confusion Matrix:")
print(confusion_matrix(y_test, y_pred))

print("\nClassification Report:")
```

```
print(classification_report(y_test, y_pred))

print("\nAccuracy Score:", accuracy_score(y_test, y_pred))
```

OUTPUT:

```
Confusion Matrix:
[[164   4]
 [ 23   9]]

Classification Report:
              precision    recall  f1-score   support

           0       0.88      0.98      0.92       168
           1       0.69      0.28      0.40        32

    accuracy                           0.86       200
   macro avg       0.78      0.63      0.66       200
weighted avg       0.85      0.86      0.84       200


Accuracy Score: 0.865
/usr/local/lib/python3.11/dist-packages/sklearn/linear_model/_logistic.py:465: ConvergenceWarning: lbfgs failed to converge (status=1):
STOP: TOTAL NO. OF ITERATIONS REACHED LIMIT.

Increase the number of iterations (max_iter) or scale the data as shown in:
    https://scikit-learn.org/stable/modules/preprocessing.html
Please also refer to the documentation for alternative solver options:
    https://scikit-learn.org/stable/modules/linear_model.html#logistic-regression
  n_iter_i = _check_optimize_result(
```

Possible Viva Questions:

Topic: Predicting ad clicks (binary classification).

1. Why is Linear Regression not suitable for binary classification problems?
2. What is the Sigmoid (or Logistic) function, and what is its range?
3. How do you interpret the output of a Logistic Regression model (probability vs. class label)?
4. What is the threshold value, and how does changing it affect precision and recall?
5. What cost function is used in Logistic Regression (Log Loss)?
6. What is a Confusion Matrix?
7. Explain Accuracy, Precision, Recall, and F1-Score.
8. What is the decision boundary in Logistic Regression? Is it linear or non-linear?
9. How can Logistic Regression be extended to multi-class classification (e.g., One-vs-Rest)?
10. What is the impact of outliers on Logistic Regression compared to Linear Regression?
11. Can Logistic Regression solve the XOR problem? Why or why not?
12. What is the difference between a False Positive and a False Negative?

6: Comparing Regression Models

Aim: Compare the performance of different regression models on a given dataset.
Question: Evaluate and compare linear regression, polynomial regression, and Ridge/Lasso regression for a prediction task of your choice.
Dataset: Use a dataset suitable for regression, potentially from previous programs.

CODE:

```
from sklearn.datasets import fetch_california_housing
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression, Ridge, Lasso
from sklearn.preprocessing import PolynomialFeatures, StandardScaler
from sklearn.metrics import mean_squared_error, r2_score
# 1. Load the dataset
boston = fetch_california_housing()
X = boston.data
y = boston.target

# 2. Split into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=0)

# 3. Standardize the features
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# 4. Linear Regression
lr = LinearRegression()
lr.fit(X_train_scaled, y_train)
y_pred_lr = lr.predict(X_test_scaled)

# 5. Polynomial Regression (degree 2)
poly = PolynomialFeatures(degree=2)
X_train_poly = poly.fit_transform(X_train_scaled)
X_test_poly = poly.transform(X_test_scaled)

pr = LinearRegression()
pr.fit(X_train_poly, y_train)
y_pred_pr = pr.predict(X_test_poly)

# 6. Ridge Regression
ridge = Ridge(alpha=1.0)
ridge.fit(X_train_scaled, y_train)
y_pred_ridge = ridge.predict(X_test_scaled)

# 7. Lasso Regression
lasso = Lasso(alpha=0.1)
lasso.fit(X_train_scaled, y_train)
y_pred_lasso = lasso.predict(X_test_scaled)

# 8. Evaluation function
def evaluate(name, y_true, y_pred):
```

```
    print(f"{name}:")
    print(f"  MSE: {mean_squared_error(y_true, y_pred):.2f}")
    print(f"  R2 : {r2_score(y_true, y_pred):.2f}")
    print("-" * 30)


# 9. Compare models
evaluate("Linear Regression", y_test, y_pred_lr)
evaluate("Polynomial Regression", y_test, y_pred_pr)
evaluate("Ridge Regression", y_test, y_pred_ridge)
evaluate("Lasso Regression", y_test, y_pred_lasso)
```

OUTPUT:

```
Linear Regression:
  MSE: 0.53
  R2 : 0.59
------------------------------
Polynomial Regression:
  MSE: 2.76
  R2 : -1.12
------------------------------
Ridge Regression:
  MSE: 0.53
  R2 : 0.59
------------------------------
Lasso Regression:
  MSE: 0.69
  R2 : 0.47
------------------------------
```

Possible Viva Questions:

Topic: Linear vs. Polynomial vs. Ridge/Lasso.

1. What is Polynomial Regression, and when would you use it over Linear Regression?
2. What is "Overfitting," and how do you detect it in regression models?
3. What is Regularization, and why is it needed?
4. What is the main difference between Ridge (L2) and Lasso (L1) regression?
5. Which regularization technique can perform feature selection (drive coefficients to zero)?
6. What is the hyperparameter λ (or α in Scikit-Learn) in Ridge/Lasso?
7. What happens to the model complexity as the degree of the polynomial increases?
8. How does the Bias-Variance tradeoff apply when comparing these models?
9. Why is feature scaling mandatory for Ridge and Lasso regression?
10. Under what condition does Ridge Regression become equivalent to Linear Regression?
11. How do you choose the optimal degree for a polynomial regression model?
12. If your dataset has more features than observations ($p > n$), which regression model is preferable?

7: Decision Tree Classification.

Aim: Visualize and interpret decision tree models.
Question: Build a decision tree classifier to predict customer churn based on service usage patterns and account information. Visualize the tree and analyze feature importance.
Dataset: You can simulate a churn dataset or search for "telecom churn" datasets.

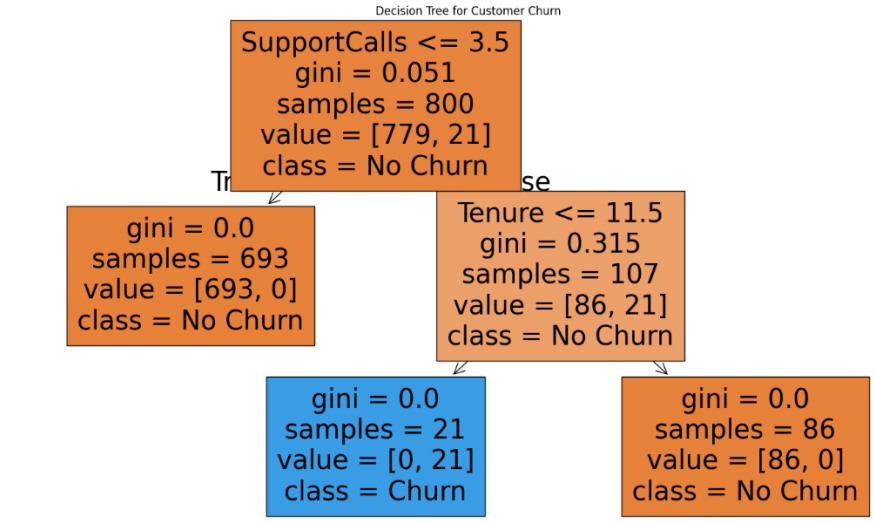
CODE:

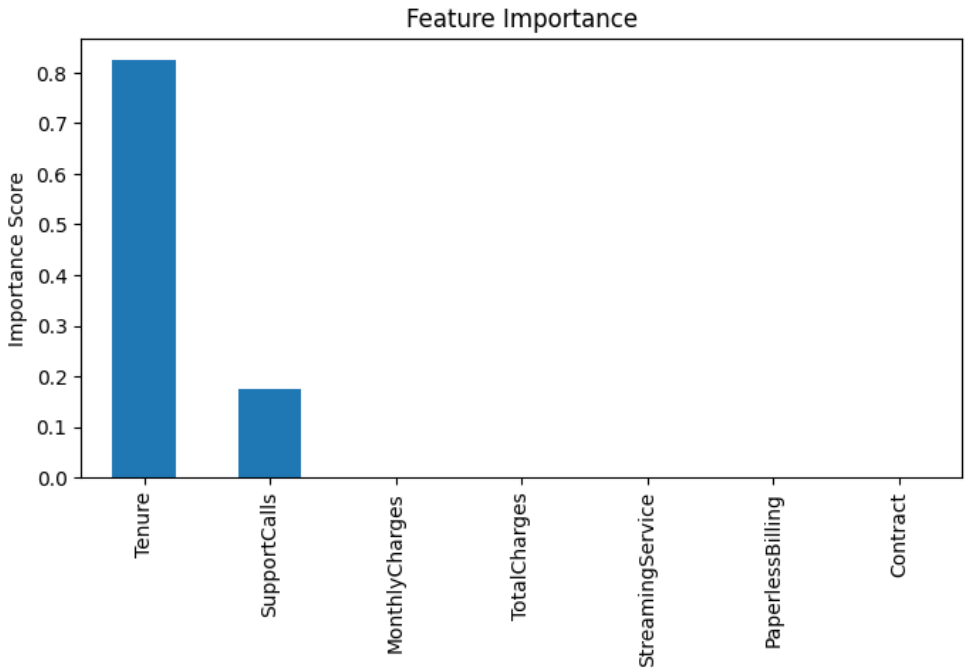
```
import numpy as np
import pandas as pd
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report, accuracy_score
import matplotlib.pyplot as plt
import seaborn as sns

# 1. Simulate Churn Dataset
np.random.seed(42)
n = 1000
data = pd.DataFrame({
    'Tenure': np.random.randint(1, 72, n),  # months
    'MonthlyCharges': np.random.uniform(20, 120, n),
    'TotalCharges': lambda df: df['Tenure'] * df['MonthlyCharges'],
    'SupportCalls': np.random.poisson(2, n),
    'StreamingService': np.random.choice([0, 1], size=n),
    'PaperlessBilling': np.random.choice([0, 1], size=n),
    'Contract': np.random.choice([0, 1, 2], size=n),  # 0=Month-to-month, 1=One year, 2=Two
year
})
data['TotalCharges'] = data['Tenure'] * data['MonthlyCharges']
# Simulate churn label (higher risk for short tenure + high support calls)
data['Churn'] = ((data['Tenure'] < 12) & (data['SupportCalls'] > 3)).astype(int)

# 2. Features and Labels
X = data.drop('Churn', axis=1)
y = data['Churn']
# 3. Train-Test Split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# 4. Train Decision Tree
clf = DecisionTreeClassifier(max_depth=4, random_state=42)
clf.fit(X_train, y_train)
# 5. Evaluate
y_pred = clf.predict(X_test)
print("Accuracy:", accuracy_score(y_test, y_pred))
print("\nClassification Report:\n", classification_report(y_test, y_pred))
# 6. Visualize the Tree
plt.figure(figsize=(18, 10))
plot_tree(clf, feature_names=X.columns, class_names=['No Churn', 'Churn'], filled=True)
plt.title("Decision Tree for Customer Churn")
plt.show()
# 7. Feature Importance
```

```
importances = pd.Series(clf.feature_importances_, index=X.columns)
importances.sort_values(ascending=False).plot(kind='bar', title='Feature Importance', figsize=(8, 4))
plt.ylabel('Importance Score')
plt.show()
```

OUTPUT:

```
Accuracy: 1.0

Classification Report:
              precision    recall  f1-score   support

           0       1.00      1.00      1.00       198
           1       1.00      1.00      1.00         2

    accuracy                           1.00       200
   macro avg       1.00      1.00      1.00       200
weighted avg       1.00      1.00      1.00       200
```





Possible Viva Questions:

Topic: Predicting customer churn.

1. What is a Decision Tree?
2. What are the Root Node, Decision Nodes, and Leaf Nodes?
3. What criteria are used to split nodes (Entropy, Gini Impurity, Information Gain)?
4. Explain the concept of "Entropy" in information theory.
5. What is "Pruning," and why is it necessary?
6. What is the difference between Pre-pruning and Post-pruning?
7. Why do Decision Trees tend to overfit the training data?
8. Can Decision Trees handle both numerical and categorical data?
9. How does a Decision Tree handle missing values (conceptually)?
10. What is the main advantage of Decision Trees regarding interpretability?
11. How does the depth of the tree affect the model's performance?
12. What is "Information Gain," and how is it calculated?

8: Ensemble Methods - Random Forest.

Aim: Apply the Random Forest algorithm and evaluate its performance.
Question: Build a Random Forest classifier for a classification task. Tune hyperparameters (e.g., number of trees) to optimize performance.
Dataset: Choose a dataset suitable for classification, possibly from a previous program.

- Build a Random Forest Classifier
- Tune hyperparameters like number of trees
- Evaluate model performance on a classification task

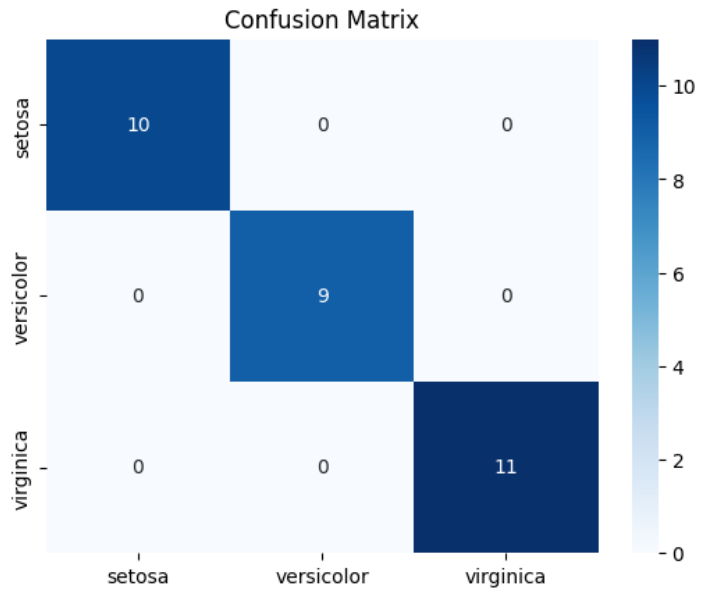
CODE:

```
import numpy as np
import pandas as pd
from sklearn.datasets import load_iris
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.metrics import classification_report, confusion_matrix, accuracy_score
import seaborn as sns
import matplotlib.pyplot as plt
# 1. Load dataset
iris = load_iris()
X = pd.DataFrame(iris.data, columns=iris.feature_names)
y = pd.Series(iris.target, name='Target')

# 2. Split data
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# 3. Random Forest with default parameters
rf = RandomForestClassifier(random_state=42)
rf.fit(X_train, y_train)
y_pred = rf.predict(X_test)

# 4. Evaluate model
print("Random Forest Classifier (Default Parameters):")
print("Accuracy:", accuracy_score(y_test, y_pred))
print("Classification Report:\n", classification_report(y_test, y_pred))

# 5. Confusion Matrix
sns.heatmap(confusion_matrix(y_test, y_pred), annot=True, fmt='d',
            xticklabels=iris.target_names, yticklabels=iris.target_names, cmap="Blues")
plt.title("Confusion Matrix")
plt.show()

# 6. Hyperparameter Tuning: GridSearch
param_grid = {
    'n_estimators': [10, 50, 100],
    'max_depth': [None, 3, 5, 10],
    'min_samples_split': [2, 4],
}

grid = GridSearchCV(RandomForestClassifier(random_state=42), param_grid, cv=5)
```

```
grid.fit(X_train, y_train)

print("Best Parameters:", grid.best_params_)
# 7. Evaluate Best Model
best_rf = grid.best_estimator_
y_pred_best = best_rf.predict(X_test)

print("\nRandom Forest Classifier (Tuned Parameters):")
print("Accuracy:", accuracy_score(y_test, y_pred_best))
print("Classification Report:\n", classification_report(y_test, y_pred_best))

# 8. Feature Importance Plot
importances = pd.Series(best_rf.feature_importances_, index=X.columns)
importances.sort_values().plot(kind='barh', title='Feature Importances')
plt.xlabel("Importance Score")
plt.show()
```

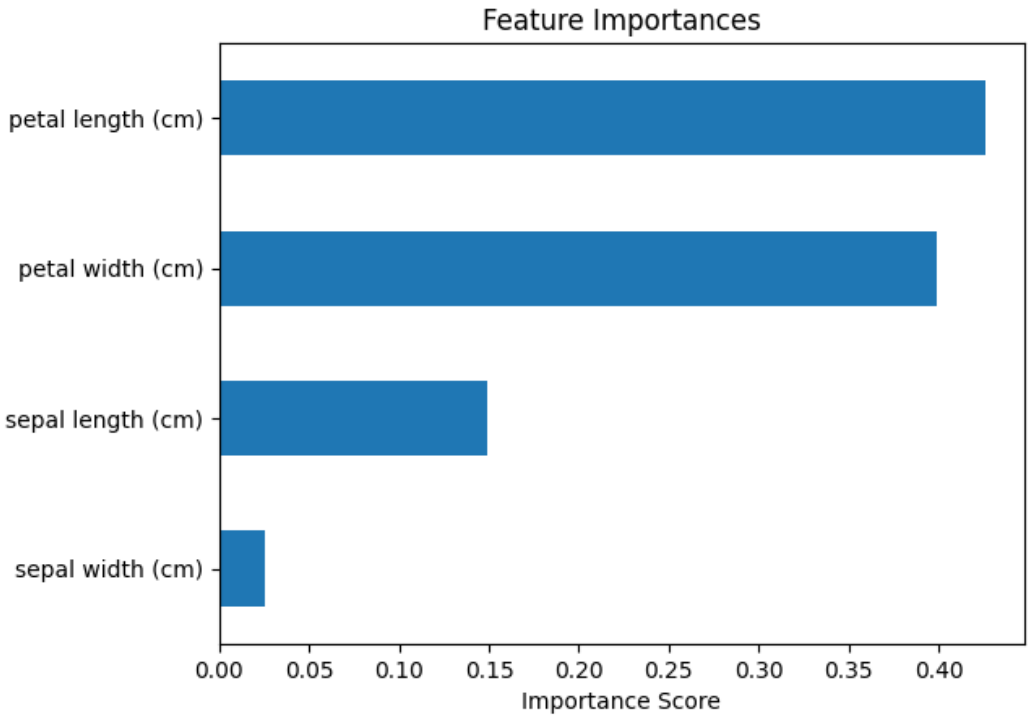
OUTPUT:

```
Random Forest Classifier (Default Parameters):
Accuracy: 1.0
Classification Report:
               precision    recall  f1-score   support

           0       1.00      1.00      1.00        10
           1       1.00      1.00      1.00         9
           2       1.00      1.00      1.00        11

    accuracy                           1.00        30
   macro avg       1.00      1.00      1.00        30
weighted avg       1.00      1.00      1.00        30
```



Best Parameters: {'max_depth': None, 'min_samples_split': 2, 'n_estimators': 10}

```
Random Forest Classifier (Tuned Parameters):
Accuracy: 1.0
Classification Report:
              precision    recall  f1-score   support

           0       1.00      1.00      1.00        10
           1       1.00      1.00      1.00         9
           2       1.00      1.00      1.00        11

    accuracy                           1.00        30
   macro avg       1.00      1.00      1.00        30
weighted avg       1.00      1.00      1.00        30
```



Possible Viva Questions:

Topic: Hyperparameter tuning and classification.

1. What is "Ensemble Learning"?
2. How does a Random Forest differ from a single Decision Tree?
3. What is "Bagging" (Bootstrap Aggregating)?
4. Why is it called a "Random" Forest? (What is randomized?)
5. Does Random Forest typically suffer from overfitting as much as a Decision Tree?
6. What is "Out-of-Bag" (OOB) error?
7. What are the important hyperparameters to tune in a Random Forest?
8. How does Random Forest determine feature importance?
9. Can Random Forest be used for regression tasks?
10. What happens if you increase the number of trees in the forest indefinitely?
11. How does the algorithm handle the "bias-variance" tradeoff compared to a single tree?
12. Is Random Forest faster to train than a Decision Tree? Why or why not?

9: Naive Bayes for Text Classification.

Aim: Implement the Naive Bayes algorithm for text data.
Question: Build a spam email classifier using the Naive Bayes algorithm.
Dataset: Spambase: https://archive.ics.uci.edu/ml/datasets/Spambase
Other text datasets: https://www.kaggle.com/datasets?tags=text

CODE:

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.naive_bayes import MultinomialNB
from sklearn.metrics import accuracy_score, classification_report

# 1. Load dataset (after downloading and placing 'spam.csv' in your folder)
df = pd.read_csv("/content/drive/MyDrive/Colab Notebooks/Datasets/spam.csv",
encoding='latin-1')[['v1', 'v2']]
df.columns = ['label', 'message']
# 2. Encode labels
df['label'] = df['label'].map({'ham': 0, 'spam': 1})

# 3. Text vectorization
vectorizer = CountVectorizer()
X = vectorizer.fit_transform(df['message'])
y = df['label']

# 4. Train/test split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# 5. Train Naive Bayes model
model = MultinomialNB()
model.fit(X_train, y_train)

# 6. Predict & evaluate
y_pred = model.predict(X_test)
print("Accuracy:", accuracy_score(y_test, y_pred))
print("\nClassification Report:\n", classification_report(y_test, y_pred))
```

OUTPUT:

```
Accuracy: 0.97847533632287

Classification Report:
               precision    recall  f1-score   support

           0       0.99      0.99      0.99       965
           1       0.91      0.93      0.92       150

    accuracy                           0.98      1115
   macro avg       0.95      0.96      0.95      1115
weighted avg       0.98      0.98      0.98      1115
```

Possible Viva Questions:

Topic: Spam email classifier.

1. What is Bayes' Theorem?
2. Why is the algorithm called "Naive"?
3. What is the independence assumption?
4. How do you convert text data into numerical format for Naive Bayes (Bag of Words, TF-IDF)?
5. What is Laplace Smoothing (Additive Smoothing), and why is it used?
6. What happens if a word in the test set was never seen in the training set (Zero Frequency Problem)?
7. What are the different types of Naive Bayes classifiers (Gaussian, Multinomial, Bernoulli)?
8. Why is Naive Bayes popular for text classification specifically?
9. Is Naive Bayes a linear or non-linear classifier?
10. How does the size of the vocabulary affect the performance?
11. Can Naive Bayes handle missing values?
12. Despite its "naive" assumption, why does it often work well in practice?

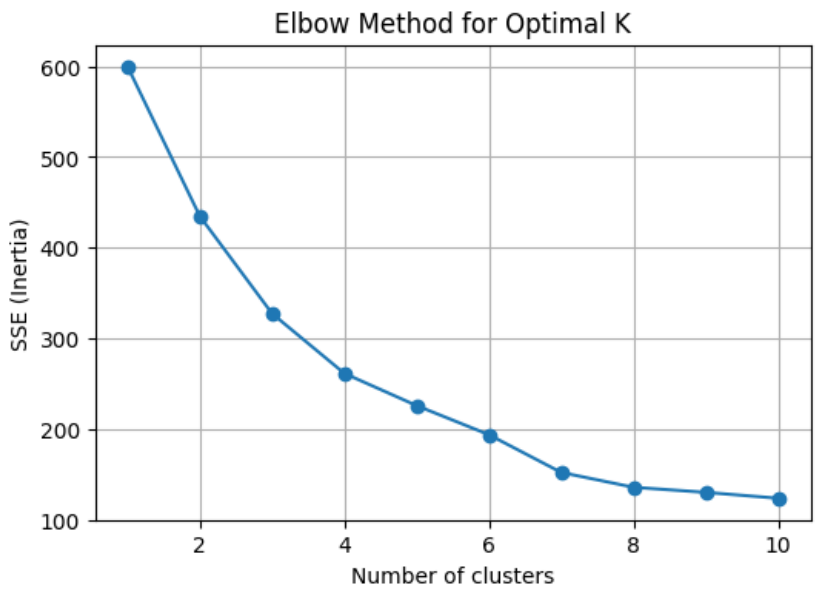
10: Customer Segmentation with K-Means Clustering.

Aim: Apply clustering to group similar data points.
Question: Segment customers based on their purchasing behavior (e.g., RFM: Recency, Frequency, Monetary value) using k-means clustering. Visualize the clusters.
Dataset: Simulate a customer purchase dataset or use a retail dataset with transaction history.

CODE:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler

# 1. Simulate customer RFM data
np.random.seed(42)
n_customers = 200
data = pd.DataFrame({
    'CustomerID': range(1, n_customers + 1),
    'Recency': np.random.randint(1, 100, size=n_customers),     # days since last purchase
    'Frequency': np.random.randint(1, 20, size=n_customers),    # number of purchases
    'Monetary': np.random.randint(100, 10000, size=n_customers) # total money spent
})

# 2. Standardize the RFM features
scaler = StandardScaler()
rfm_scaled = scaler.fit_transform(data[['Recency', 'Frequency', 'Monetary']])

# 3. Find optimal number of clusters using the Elbow Method
sse = []
for k in range(1, 11):
    kmeans = KMeans(n_clusters=k, random_state=42)
    kmeans.fit(rfm_scaled)
    sse.append(kmeans.inertia_)

# Plot Elbow Curve
plt.figure(figsize=(6, 4))
plt.plot(range(1, 11), sse, marker='o')
plt.title("Elbow Method for Optimal K")
plt.xlabel("Number of clusters")
plt.ylabel("SSE (Inertia)")
plt.grid(True)
plt.show()
# 4. Apply K-Means with optimal clusters (e.g., k=4)
kmeans = KMeans(n_clusters=4, random_state=42)
data['Cluster'] = kmeans.fit_predict(rfm_scaled)


# 5. Visualize Clusters
plt.figure(figsize=(8, 6))
sns.scatterplot(data=data, x='Recency', y='Monetary', hue='Cluster', palette='Set2')
```

```
plt.title("Customer Segmentation (Recency vs Monetary)")
plt.show()

plt.figure(figsize=(8, 6))
sns.scatterplot(data=data, x='Frequency', y='Monetary', hue='Cluster', palette='Set1')
plt.title("Customer Segmentation (Frequency vs Monetary)")
plt.show()
```



Possible Viva Questions:

Topic: Segmentation using RFM values.

1. What is the difference between Supervised and Unsupervised learning?
2. How does the K-Means algorithm work (step-by-step)?

3. What does 'K' represent in K-Means?
4. How do you determine the optimal number of clusters (Elbow Method)?
5. What is the "Silhouette Score"?
6. What is the objective function of K-Means (Inertia/Within-Cluster Sum of Squares)?
7. Why is random initialization of centroids a problem, and what is K-Means++?
8. How does K-Means assign a point to a cluster?
9. Is K-Means guaranteed to find the global optimum?
10. Why is feature scaling critical for K-Means?
11. What is the shape of the clusters formed by K-Means?
12. How would you interpret the clusters found in your customer segmentation task?